Puente: de MLE a variables latentes

Algoritmo EM: Mixtura de gaussianas

Algoritmo EM: caso general

PCA: variable latente continua

ACM: variable latente categorica

Sintesis y cierre

Referencias

Análisis Avanzado de Datos

Sesión 6: Variables Latentes

Nicolás López

Primer semestre de 2026

1. Puente: de MLE a variables latentes
2. Algoritmo EM: Mixtura de gaussianas
3. Algoritmo EM: caso general
4. PCA: variable latente continua
5. ACM: variable latente categorica
6. Sintesis y cierre
7. Referencias

Puente: de MLE a variables latentes

Resumen de la Sesion 5

En la Sesion 5 vimos que los modelos estadistico bajo máxima verosimilitud tienen la misma estructura:

$$\hat{\theta}_{\text{MLE}} = \underset{\theta}{\text{argmax}}\ L(\theta \mid y_1, \ldots, y_n) = \underset{\theta}{\text{argmax}} \sum_i \log f(y_i \mid \theta)$$

- Para RLM: $Y_i \sim N(\beta_0 + \beta_1 x_i, \sigma)$, con solucion analitica $(X^T X)^{-1} X^T y$.
- Para regresion logistica: $Y_i \sim \text{Ber}(\pi(x_i))$, con solucion numerica.

En ambos casos, **todos los datos** (x_i, y_i) son observados directamente.

La pregunta de hoy

Que ocurre cuando hay informacion relevante para el modelo que **no podemos observar**?

Ejemplos cotidianos:

- Un histograma bimodal sugiere dos subpoblaciones, pero no sabemos a cual pertenece cada dato.
- Tenemos 10 variables correlacionadas y sospechamos que unas pocas dimensiones "latentes" las explican.
- Una encuesta tiene 20 preguntas categoricas y queremos resumir el patron de respuestas.

Variable latente

Una variable que **existe** y **determina** los datos observados, pero que **no podemos medir** directamente.

El marco unificador de esta sesion

Los tres metodos de esta sesion comparten la misma logica: hay una **variable latente** Z que no vemos, pero que estructura los datos observados X.

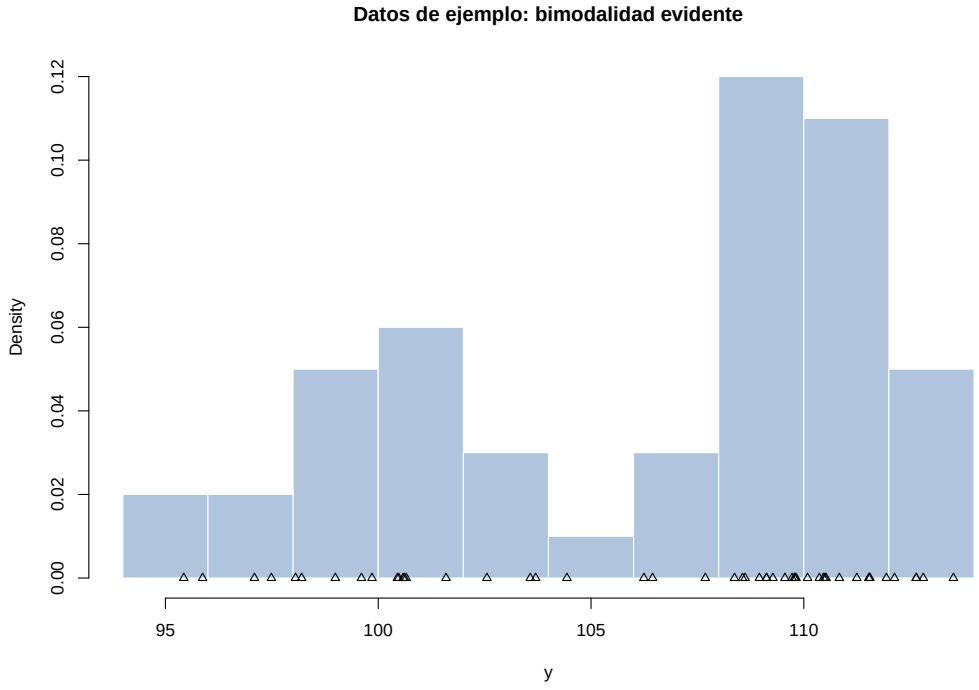
Metodo	Datos X	Variable latente Z	Objetivo
EM/Mixtura	Continuos	Membresia al grupo ($\Delta_i \in \{0, 1\}$)	Estimar grupos
PCA	Continuos, p variables	Scores en $k \ll p$ dims.	Reducir dimension
ACM	Categoricos	Scores en k dims.	Reducir dimension

En todos los casos, descubrir Z es el objetivo. La diferencia es el tipo de dato y la estructura del modelo.

Algoritmo EM: Mixtura de gaussianas

El problema: datos con estructura oculta

Contamos con $n = 50$ observaciones continuas $y_1, \ldots, y_n$.



La bimodalidad sugiere **dos subpoblaciones**. Pero no sabemos a cual pertenece cada observacion. La membresia Δ_i es la **variable latente**.

El modelo de mixtura

Si conocieramos la membresia, tendriamos:

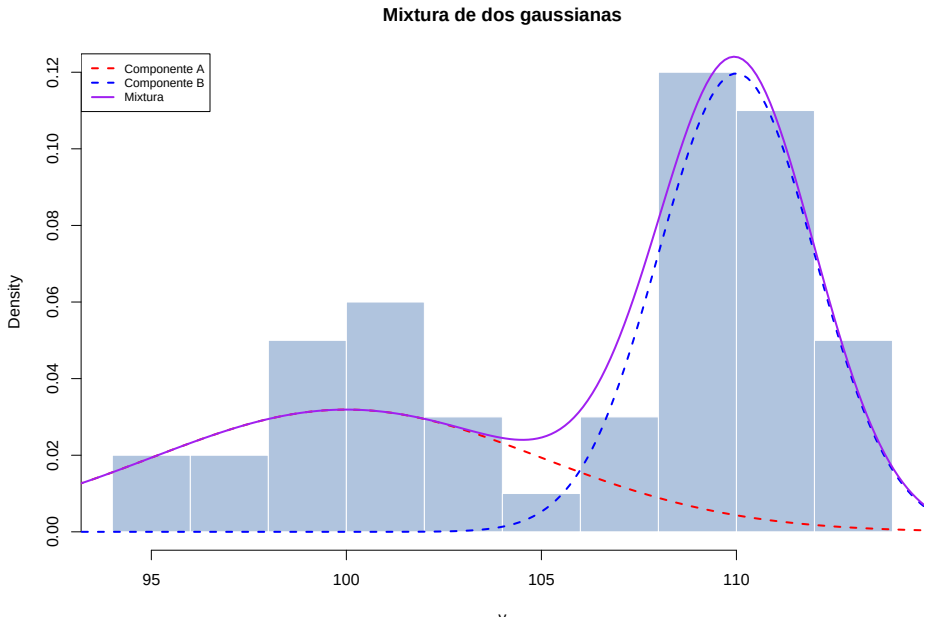
$$Y_A \sim N(\mu_1, \sigma_1) \qquad Y_B \sim N(\mu_2, \sigma_2)$$

La variable latente $\Delta_i \in \{0, 1\}$ indica a cual subpoblacion pertenece la observacion i:

$$P(\Delta_i = 1) = \pi$$

La densidad **marginal** (integrando la variable latente) es:

$$f_Y(y) = (1 - \pi)\, f_A(y|\theta_1) + \pi\, f_B(y|\theta_2)$$



La dificultad: log-verosimilitud con suma bajo el log

Los parametros a estimar son: $\theta = (\mu_1, \sigma_1, \mu_2, \sigma_2, \pi)$

La log-verosimilitud de la mixtura es:

$$\ell(\theta \mid Y) = \sum_{i=1}^{n} \log[(1-\pi)\, f_A(y_i|\theta_1) + \pi\, f_B(y_i|\theta_2)]$$

El problema es la suma **dentro** del logaritmo. Para entender por que eso complica las cosas, compara:

$$\log(a+b) \qquad \text{vs.} \qquad \log(a)+\log(b)$$

- El lado derecho es facil: la derivada respecto a los parametros de a y b se calcula por separado.
- El lado izquierdo mezcla los parametros de ambas componentes dentro de un mismo logaritmo: no hay forma de separar la optimizacion sobre θ_1 de la optimizacion sobre θ_2.

En particular, derivar e igualar a cero produce ecuaciones sin solucion analitica.

La dificultad: ejemplo numerico

Si $n = 3$ observaciones y $\pi = 0.5$, hay que maximizar:

$$\ell = \log[.5\, f_A(y_1) + .5\, f_B(y_1)] + \log[.5\, f_A(y_2) + .5\, f_B(y_2)] + \log[.5\, f_A(y_3) + .5\, f_B(y_3)]$$

Para encontrar $\hat{\mu}_1$ (la media de la componente A), la derivada respecto a μ_1 involucra terminos que dependen simultaneamente de μ_1, σ_1, μ_2, σ_2 y π a la vez. El sistema de ecuaciones resultante no tiene solucion cerrada.

La idea del algoritmo EM: si conocieramos $\Delta_1, \ldots, \Delta_n$, la verosimilitud se simplificaria enormemente. Como no los conocemos, los **estimamos** en cada iteracion.

La log-verosimilitud completa

Asumamos $\Delta_T = (\Delta_1, \ldots, \Delta_n)$ conocidos. La log-verosimilitud **completa** (datos + latentes) es:

$$\ell_0(\theta \mid Y, \Delta) = \sum_i (1 - \Delta_i) \log[(1 - \pi) f_A(y_i|\theta_1)] + \Delta_i \log[\pi\, f_B(y_i|\theta_2)]$$

Por que ahora si hay solucion cerrada?

La clave: el logaritmo ya no envuelve una suma. Cada termino de la suma exterior solo contiene **una de las dos componentes** (gracias a Δ_i).

Se puede reescribir como:

$$\ell_0 = \underbrace{\sum_{i:\Delta_i=0} \log[(1-\pi) f_A(y_i|\theta_1)]}_{\text{solo depende de } \theta_1, \pi} + \underbrace{\sum_{i:\Delta_i=1} \log[\pi\, f_B(y_i|\theta_2)]}_{\text{solo depende de } \theta_2, \pi}$$

Ahora los parametros de A y B estan **separados**. Se puede maximizar sobre θ_1 ignorando θ_2 y viceversa. Cada sub-problema tiene solucion analitica:

- $\hat{\mu}_j$: media de los y_i asignados al grupo j.
- $\hat{\sigma}_j^2$: varianza de los y_i del grupo j.
- $\hat{\pi}$: proporcion de observaciones con $\Delta_i = 1$.

La responsabilidad: derivacion con Bayes

Queremos $P(\Delta_i = 1 \mid y_i, \theta)$: dado que observamos y_i, que tan probable es que venga de B?

Aplicamos el **teorema de Bayes**:

$$P(\Delta_i = 1 \mid y_i, \theta) = \frac{P(y_i \mid \Delta_i = 1, \theta) \cdot P(\Delta_i = 1 \mid \theta)}{P(y_i \mid \theta)}$$

Identificando cada termino:

- $P(\Delta_i = 1 \mid \theta) = \hat{\pi}$: probabilidad a priori de ser del grupo B.
- $P(y_i \mid \Delta_i = 1, \theta) = f_B(y_i|\hat{\theta}_2)$: densidad de B evaluada en y_i.
- $P(y_i \mid \theta) = (1 - \hat{\pi}) f_A(y_i|\hat{\theta}_1) + \hat{\pi} f_B(y_i|\hat{\theta}_2)$: probabilidad total por la regla de la probabilidad total.

Sustituyendo:

$$\hat{\gamma}_i = \frac{\hat{\pi}\, f_B(y_i|\hat{\theta}_2)}{(1 - \hat{\pi}) f_A(y_i|\hat{\theta}_1) + \hat{\pi}\, f_B(y_i|\hat{\theta}_2)}$$

$\hat{\gamma}_i \in [0, 1]$ es la **responsabilidad**: la probabilidad posterior de que y_i provenga de la componente B.

Del log-likelihood completo al Paso E

Con Δ_i conocido (log-verosimilitud completa):

$$\ell_0 = \sum_i \Big[(1-\Delta_i)\log[(1-\pi)f_A(y_i|\theta_1)] + \Delta_i \log[\pi\, f_B(y_i|\theta_2)]\Big]$$

Como Δ_i no es observable, se reemplaza por su valor esperado dado los parametros actuales y los datos:

$$E[\Delta_i \mid y_i, \theta^{(t)}] = \hat{\gamma}_i$$

Paso E: tomar esperanza respecto a Δ_T:

$$Q(\theta, \theta^{(t)}) = E_{\Delta|Y,\theta^{(t)}}[\ell_0]$$

$$= \sum_i \Big[(1-\hat{\gamma}_i)\log[(1-\pi)f_A(y_i|\theta_1)] + \hat{\gamma}_i \log[\pi\, f_B(y_i|\theta_2)]\Big]$$

Se reemplaza cada Δ_i por su responsabilidad $\hat{\gamma}_i$: los Δ_i duros (0 o 1) se convierten en pesos suaves ($\in [0,1]$).

Paso M: actualizar los parametros

Dado $\hat{\gamma}_i$ para $i = 1, \ldots, n$, se actualizan los parametros maximizando la log-verosimilitud esperada:

Para la componente A (peso $1 - \hat{\gamma}_i$):

$$\hat{\mu}_1 = \frac{\sum_i (1 - \hat{\gamma}_i)\, y_i}{\sum_i (1 - \hat{\gamma}_i)} \qquad \hat{\sigma}_1^2 = \frac{\sum_i (1 - \hat{\gamma}_i)(y_i - \hat{\mu}_1)^2}{\sum_i (1 - \hat{\gamma}_i)}$$

Para la componente B (peso $\hat{\gamma}_i$):

$$\hat{\mu}_2 = \frac{\sum_i \hat{\gamma}_i\, y_i}{\sum_i \hat{\gamma}_i} \qquad \hat{\sigma}_2^2 = \frac{\sum_i \hat{\gamma}_i (y_i - \hat{\mu}_2)^2}{\sum_i \hat{\gamma}_i}$$

Para la mezcla:

$$\hat{\pi} = \frac{\sum_i \hat{\gamma}_i}{n}$$

El algoritmo EM: resumen

Algoritmo EM para mixtura de gaussianas

1. **Inicializar:** $\hat{\mu}_j = y_{\text{rand}}$, $\hat{\sigma}_j = \hat{\sigma}_{\text{global}}$, $\hat{\pi} = 0.5$.
2. **Paso E:** calcular $\hat{\gamma}_i$ para $i = 1, \ldots, n$ con los parametros actuales.
3. **Paso M:** actualizar $(\hat{\mu}_1, \hat{\sigma}_1, \hat{\mu}_2, \hat{\sigma}_2, \hat{\pi})$ dados $\hat{\gamma}_i$.
4. Repetir pasos 2–3 hasta convergencia ($|\ell^{(t+1)} - \ell^{(t)}| < \varepsilon$).

Por que la verosimilitud nunca decrece?

La intuicion: en cada iteracion hacemos dos cosas.

Paso E: dado el θ actual, calculamos la mejor asignacion posible de responsabilidades. Esto no cambia θ, por lo que ℓ tampoco cambia.

Paso M: dado $\hat{\gamma}_i$, encontramos el θ que **maximiza** la log-verosimilitud esperada. Esto garantiza que el nuevo $\theta^{(t+1)}$ produce una verosimilitud al menos tan alta como $\theta^{(t)}$.

Formalmente:

$$\ell(\theta^{(t+1)} \mid Y) \geq \ell(\theta^{(t)} \mid Y)$$

La log-verosimilitud sube (o se queda igual) en cada iteracion, como subir escalones: nunca bajamos.

Esto garantiza **convergencia**, pero solo hacia el escalon mas alto que sea alcanzable desde el punto de inicio: un maximo **local**, no necesariamente el global.

Relacion entre las dos verosimilitudes

Existe una ecuacion exacta que conecta la log-verosimilitud observada con la completa. Para entenderla, partimos de:

$$P(y_i, \Delta_i \mid \theta) = P(y_i \mid \theta)\, P(\Delta_i \mid y_i, \theta)$$

Tomando logaritmo:

$$\log P(y_i, \Delta_i \mid \theta) = \log P(y_i \mid \theta) + \log P(\Delta_i \mid y_i, \theta)$$

Que se puede reordenar como:

$$\underbrace{\log P(y_i \mid \theta)}_{\text{lo que queremos}} = \underbrace{\log P(y_i, \Delta_i \mid \theta)}_{\ell_0\text{: facil}} - \underbrace{\log P(\Delta_i \mid y_i, \theta)}_{\text{incertidumbre sobre } \Delta_i}$$

Interpretacion de la relacion

Sumando sobre todos los i y tomando esperanza respecto a Δ_T:

$$\ell(\theta \mid Y) = \underbrace{Q(\theta, \theta^{(t)})}_{\text{lo que maximizamos en M}} - \underbrace{H(\theta, \theta^{(t)})}_{\text{termino de entropia}}$$

Para que sirve esta relacion:

- Muestra que maximizar Q (Paso M) es correcto: el termino H se puede demostrar que **no decrece** al pasar de $\theta^{(t)}$ a $\theta^{(t+1)}$, lo que garantiza que $\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$.
- Proporciona la base teorica de la convergencia del algoritmo.
- En la practica: nos dice que el algoritmo EM esta bien construido matematicamente, aunque solo maximicemos Q (que es la parte facil) en lugar de ℓ directamente.

Implementacion en R

```
library(mixtools)
set.seed(1)
init_mu <- sample(length(dat), 2)
gm <- normalmixEM(dat, k=2,
                  lambda = c(0.5, 0.5),
                  mu     = c(dat[init_mu[1]],
                             dat[init_mu[2]]),
                  sigma  = c(sd(dat), sd(dat)))
```

```
## number of iterations= 28
```

```
cat("Pesos (pi):  ", round(gm$lambda, 3), "\n")
```

```
## Pesos (pi):   0.394 0.606
```

```
cat("Medias:      ", round(gm$mu, 3),     "\n")
```

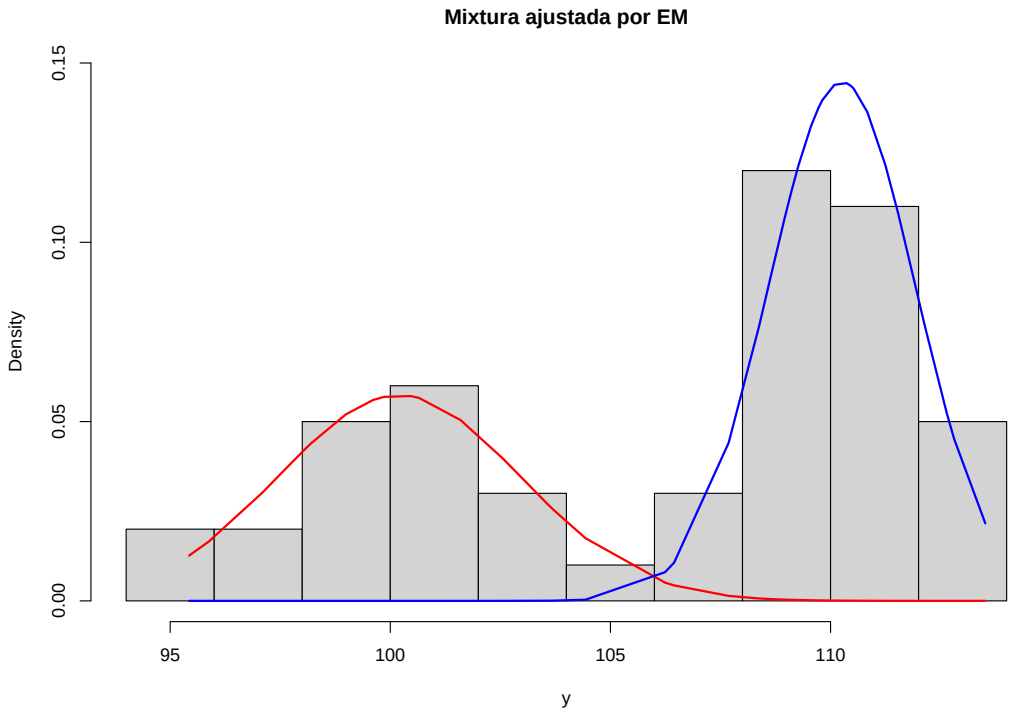
```
## Medias:       100.203 110.259
```

```
cat("Desv. est.:  ", round(gm$sigma, 3),  "\n")
```

```
## Desv. est.:   2.744 1.67
```

El modelo recupera $\mu_1 \approx 100$, $\mu_2 \approx 110$ y proporciones ≈ 0.40 y 0.60: muy cercano a los parametros verdaderos usados para simular los datos.

Densidad ajustada por EM



Recuperacion de la variable latente

La responsabilidad $\hat{\gamma}_i$ permite asignar cada observacion a su subpoblacion mas probable:

```
# Asignacion por maxima responsabilidad
asign <- apply(gm$posterior, 1, which.max)
real  <- ifelse(1:50 < 21, "A (real)", "B (real)")
table(paste0("Grupo ",asign," (EM)"), real)
```

```
##                 real
##                  A (real) B (real)
##   Grupo 1 (EM)       19        0
##   Grupo 2 (EM)        1       30
```

La variable latente Δ_i fue recuperada correctamente para la gran mayoria de observaciones: EM descubrio la estructura oculta a partir solo de los datos observados.

Mixtura gaussiana: resumen del ciclo EM

Veamos como fluye una iteracion completa con numeros concretos. Supongamos $\theta^{(0)} : \mu_1 = 95, \mu_2 = 115, \sigma_1 = \sigma_2 = 5, \pi = 0.5$.

Paso E: para cada y_i calculamos:

$$\hat{\gamma}_i = \frac{0.5 \cdot \phi(y_i; 115, 5)}{0.5 \cdot \phi(y_i; 95, 5) + 0.5 \cdot \phi(y_i; 115, 5)}$$

Un punto en $y = 100$: $\hat{\gamma} \approx 0.04$ (casi seguro componente A). Un punto en $y = 107$: $\hat{\gamma} \approx 0.50$ (ambiguo, a medio camino). Un punto en $y = 112$: $\hat{\gamma} \approx 0.88$ (casi seguro componente B).

Paso M: con esas responsabilidades actualizamos:

$$\hat{\mu}_1 = \frac{\sum_i (1 - \hat{\gamma}_i) y_i}{\sum_i (1 - \hat{\gamma}_i)} \quad \longrightarrow \quad \text{media ponderada de puntos cerca de A}$$

$$\hat{\mu}_2 = \frac{\sum_i \hat{\gamma}_i y_i}{\sum_i \hat{\gamma}_i} \quad \longrightarrow \quad \text{media ponderada de puntos cerca de B}$$

Las medias se acercan a los centros reales. Se repite hasta convergencia.

Puente: de MLE a variables latentes

Algoritmo EM: Mixtura de gaussianas

Algoritmo EM: caso general

PCA: variable latente continua

ACM: variable latente categorica

Sintesis y cierre

Referencias

Algoritmo EM: caso general

Formulacion general

El problema de mixtura es un caso particular. El EM se aplica a cualquier modelo con datos incompletos:

Configuracion general del EM

- **Datos observados** Z_T: log-verosimilitud $\ell(\theta \mid Z_T)$ dificil de maximizar.
- **Datos aumentados** (Z_T, Δ_T): log-verosimilitud completa $\ell_0(\theta \mid Z_T, \Delta_T)$ mas facil de maximizar.
- Δ_T: variables latentes (no observadas).

La relacion entre ambas:

$$\ell(\theta \mid Z_T) = \ell_0(\theta \mid Z_T, \Delta_T) - \log P(\Delta_T \mid Z_T, \theta)$$

Los dos pasos en forma general

Paso E (*Expectation*): calcular la esperanza de la log-verosimilitud completa respecto a las variables latentes, dado θ actual y datos observados:

$$Q(\theta, \theta^{(t)}) = E_{\Delta_T | Z_T, \theta^{(t)}} \left[\ell_0(\theta \mid Z_T, \Delta_T) \right]$$

Paso M (*Maximization*): encontrar el θ que maximiza Q:

$$\theta^{(t+1)} = \underset{\theta}{\operatorname{argmax}}\ Q(\theta, \theta^{(t)})$$

En la mixtura gaussiana: - Paso E: $Q = E[\ell_0] \leftrightarrow$ calcular $\hat{\gamma}_i$. - Paso M: argmax $Q \leftrightarrow$ medias y varianzas ponderadas.

Convergencia y propiedades

Monotonia de la verosimilitud:

$$\ell(\theta^{(t+1)} \mid Z_T) \geq \ell(\theta^{(t)} \mid Z_T)$$

La log-verosimilitud observada nunca decrece. Esto garantiza convergencia, pero **solo a un maximo local**.

Estrategia practica:

- Correr EM con multiples inicializaciones aleatorias.
- Quedarse con la solucion de mayor verosimilitud final.

Aplicaciones mas alla de la mixtura

- Modelos de variables latentes (factor analysis).
- Modelos con datos faltantes.
- Modelos de Markov ocultos (HMM).
- Mezclas de regresiones (modelos de regresion por segmento).

Puente: de MLE a variables latentes

Algoritmo EM: Mixtura de gaussianas

Algoritmo EM: caso general

PCA: variable latente continua

ACM: variable latente categorica

Sintesis y cierre

Referencias

PCA: variable latente continua

Motivacion: el problema de la alta dimension

Tenemos p variables continuas correlacionadas. Visualizar, modelar e interpretar datos de alta dimension es costoso y dificil.

Hipotesis de PCA: existe un conjunto pequeno de **dimensiones latentes** $Z_1, \ldots, Z_k$ con $k \ll p$ que capturan la mayor parte de la variabilidad de los datos.

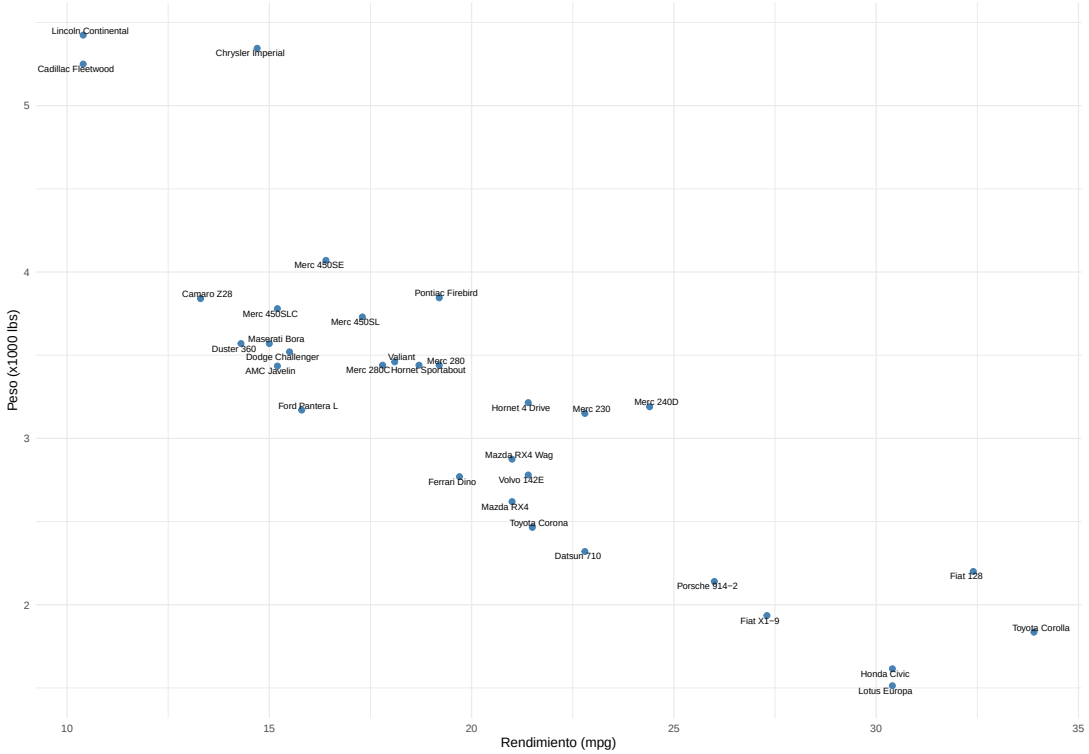
PCA como variable latente

Igual que en EM, hay algo que no observamos directamente: las **componentes principales** Z_j son combinaciones lineales de las variables originales que maximizan la varianza capturada.

Esta es exactamente la misma logica: X (observado) esta determinado por Z (latente), con la diferencia de que aqui Z es continuo y no hay estructura de grupos.

El conjunto de datos: caracteristicas de vehiculos

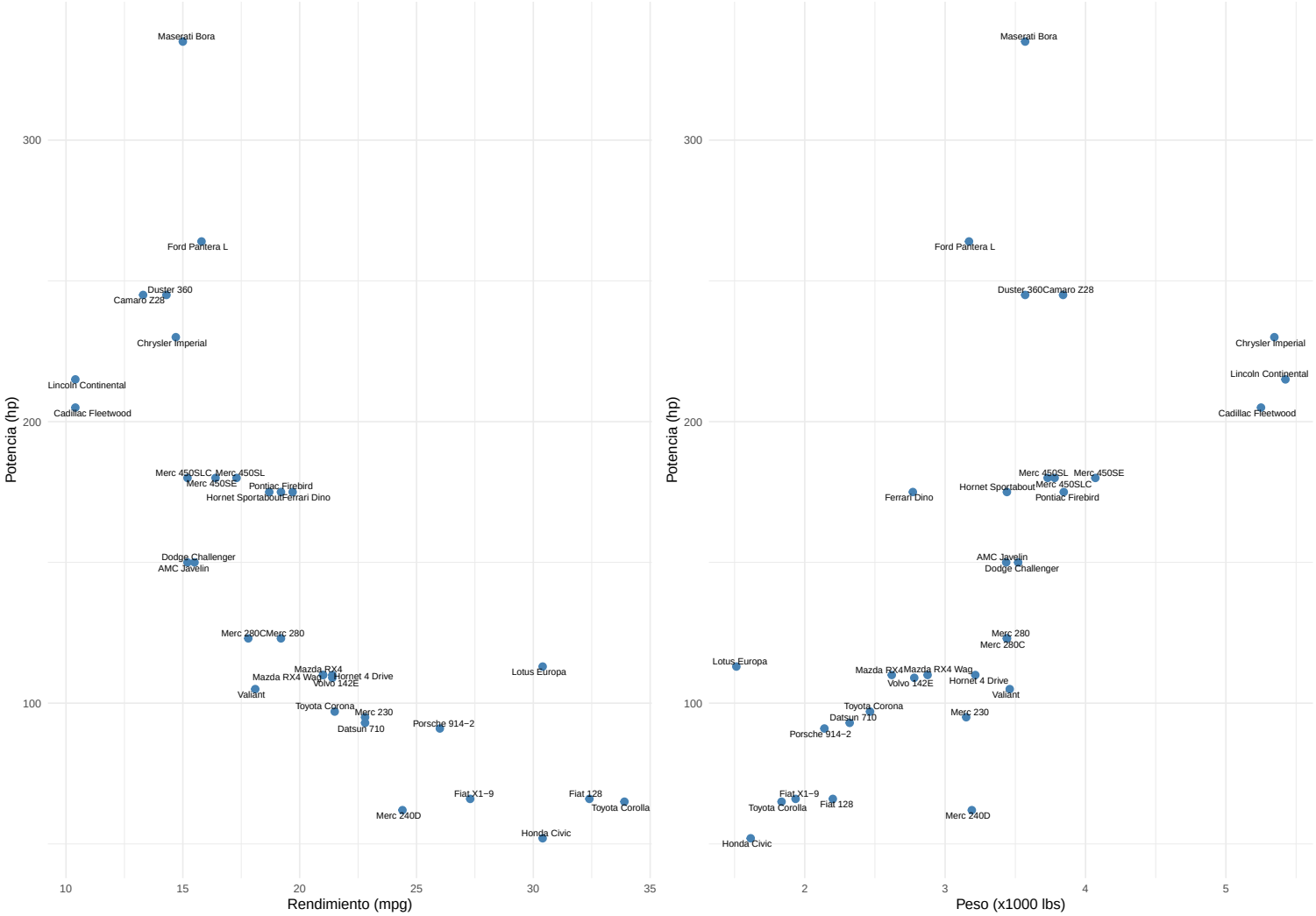
El dataset `mtcars` tiene 11 variables para 32 vehiculos.



Con 2 variables ya aparece estructura: livianos eficientes vs. pesados ineficientes.
Con $p = 11$ variables, necesitamos una representacion resumida.

El problema del par-a-par

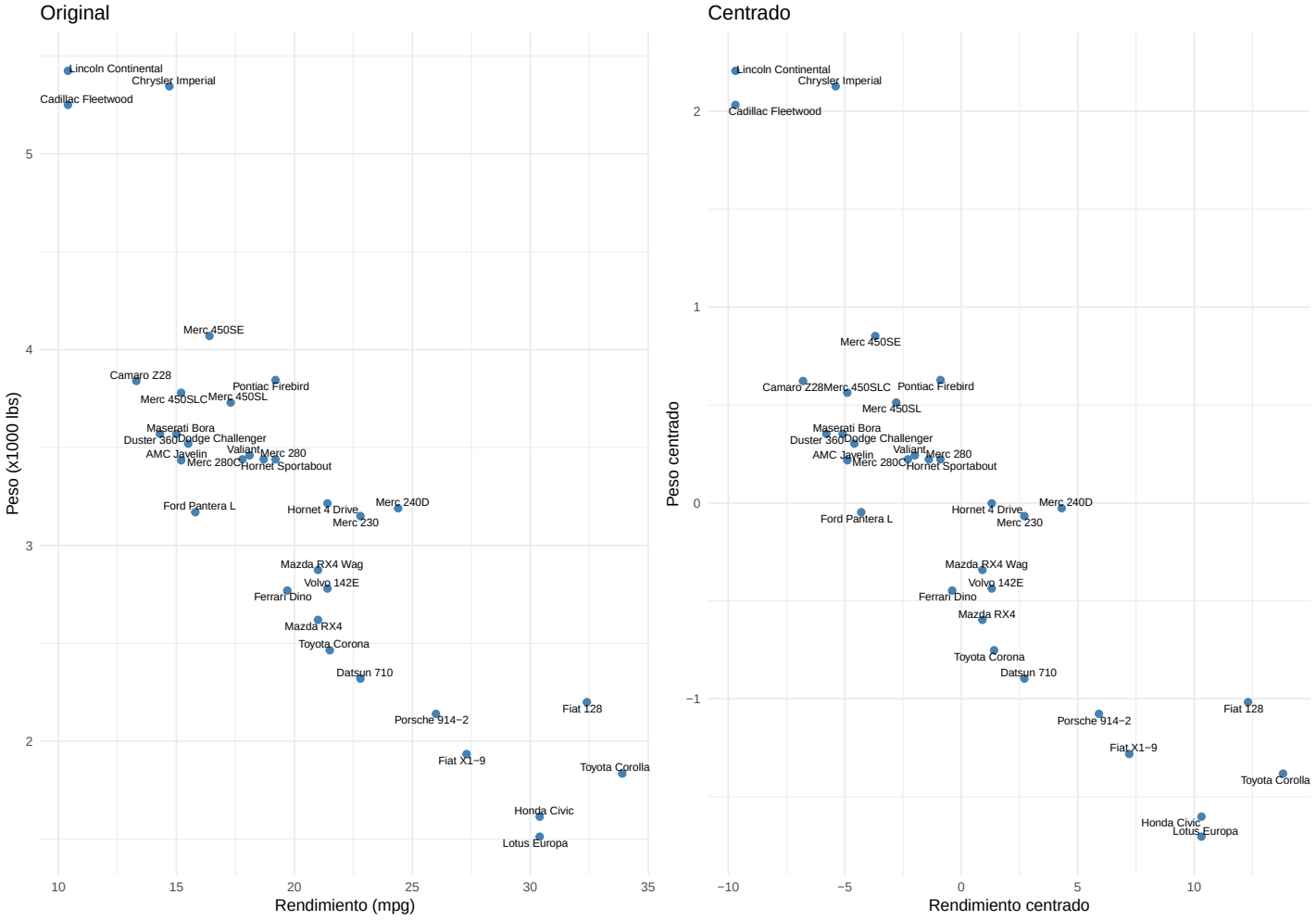
Con 3 variables ya son 3 graficos de dispersion distintos:



Con p variables hay $\binom{p}{2} = p(p-1)/2$ pares posibles. Para $p = 11$: 55 graficos. PCA los resume en 1 o 2.

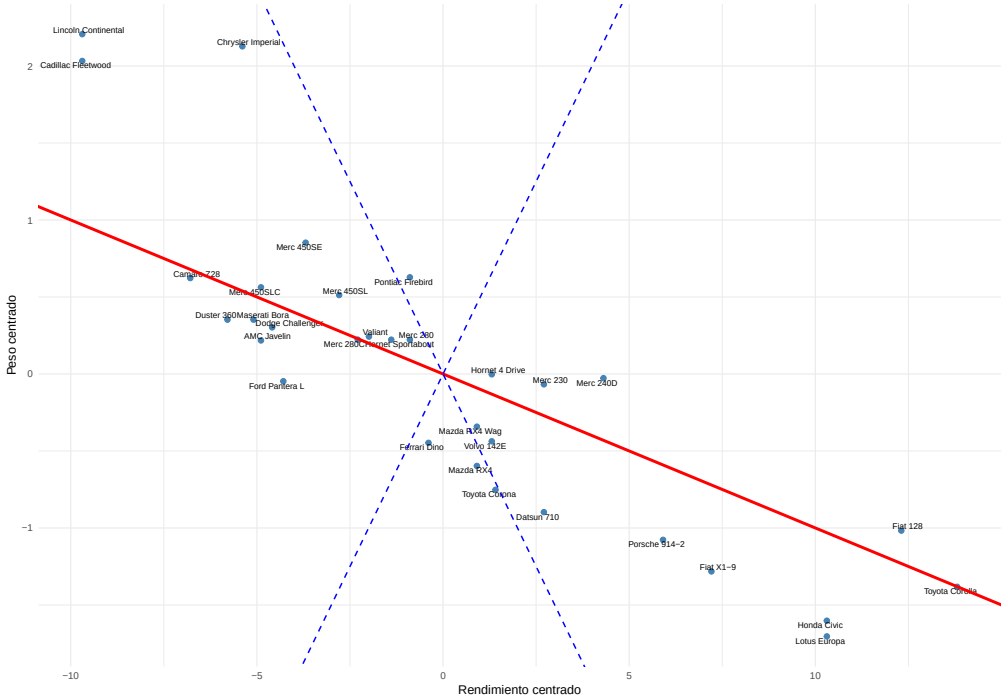
Centrado de los datos

PCA requiere datos centrados (media cero en cada variable). El centrado elimina diferencias de escala de ubicacion:



El primer componente principal: geometria

Se busca la **linea que mejor resume los datos**, es decir, la direccion w que minimiza la distancia de cada punto a su proyeccion sobre esa linea:



La linea roja (PC1, pendiente ≈ -0.1) minimiza las distancias cuadraticas de los puntos a sus proyecciones. Equivalentemente, **maximiza la varianza proyectada**.

Derivacion matematica: notacion

Sea $w \in \mathbb{R}^p$ un vector unitario ($\|w\| = 1$) que representa una direccion en el espacio de las p variables.

Notacion clave: - $x_i \in \mathbb{R}^p$: observacion i, un **vector** con las p variables medidas para el individuo i. - $X \in \mathbb{R}^{n \times p}$: **matriz** de datos con n filas (individuos) y p columnas (variables). La fila i es ${x_i}^T$.

La proyeccion: el score del individuo i en la direccion w es el producto interno $w^T x_i \in \mathbb{R}$, un escalar que mide cuanto "apunta" x_i en la direccion w.

Criterio de PCA:

$$w^* = \underset{w:\ \|w\|=1}{\operatorname{argmax}} \operatorname{Var}(w^T x_1, \ldots, w^T x_n)$$

Buscamos la direccion w que maximiza la varianza de los scores. Esa direccion captura la mayor informacion posible.

Desarrollo matricial: paso a paso

Queremos expresar $\mathrm{Var}(w^T x_1, \ldots, w^T x_n)$ en forma matricial.

Paso 1: Cada score individual es $w^T x_i \in \mathbb{R}$. Apilando los n scores en un vector:

$$\begin{pmatrix} w^T x_1 \\ w^T x_2 \\ \vdots \\ w^T x_n \end{pmatrix} = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{pmatrix} w = Xw \in \mathbb{R}^n$$

Es decir, $(w^T x_1, \ldots, w^T x_n)$ son exactamente las entradas de Xw: la multiplicacion matricial apila todos los productos internos a la vez.

Paso 2: Como los datos estan centrados, la media de los scores es cero, y la varianza de $(w^T x_1, \ldots, w^T x_n)$ es:

$$\mathrm{Var}(w^T x_1, \ldots, w^T x_n) = \frac{1}{n} \sum_i (w^T x_i)^2 = \frac{1}{n}(Xw)^T(Xw) = \frac{1}{n} w^T X^T X w$$

Paso 3: $\frac{1}{n}X^T X = \mathrm{Cov}(X)$. Por lo tanto:

$$\mathrm{Var}(w^T x_1, \ldots, w^T x_n) = w^T \mathrm{Cov}(X)\, w$$

La restriccion de Lagrange

El problema de optimizacion es:

$$w^* = \underset{w:\, w^T w = 1}{\operatorname{argmax}} \; w^T \operatorname{Cov}(X)\, w$$

Sin la restriccion $\|w\| = 1$, se podria hacer w arbitrariamente grande: la restriccion garantiza una solucion unica (salvo signo).

Usando multiplicadores de Lagrange:

$$L(w, \lambda) = w^T \operatorname{Cov}(X)\, w - \lambda(w^T w - 1)$$

Derivando respecto a w e igualando a cero:

$$\frac{\partial L}{\partial w} = 2 \operatorname{Cov}(X)\, w - 2\lambda w = 0 \quad \Rightarrow \quad \operatorname{Cov}(X)\, w = \lambda w$$

La solucion: vectores propios de Cov(X)

La ecuacion $\mathrm{Cov}(X)\, w = \lambda w$ es exactamente la definicion de **vector propio** de $\mathrm{Cov}(X)$ con valor propio λ.

La varianza capturada en la direccion w es:

$$w^T \mathrm{Cov}(X)\, w = w^T(\lambda w) = \lambda \|w\|^2 = \lambda$$

Para **maximizar la varianza capturada**, elegimos el vector propio w_1 asociado al **mayor valor propio** λ_1.

Interpretacion geometrica: $\mathrm{Cov}(X)$ tiene p pares (λ_j, w_j). Los vectores propios son las p direcciones ortogonales de maximo estiramiento de la nube de datos; los valores propios miden cuanto se estira en cada direccion.

El PCj es la j-esima de esas direcciones.

Interpretacion de variable latente

El score del individuo i en el PCj es $z_{ij} = w_j^T x_i$: una nueva variable que no existia en los datos originales. Es la proyeccion de x_i sobre la j-esima dimension de maxima variacion.

La importancia de estandarizar

Si las variables tienen distintas escalas, las de mayor varianza dominan el PCA.

Solucion: usar `scale=TRUE` en `prcomp()`, que divide cada variable por su desviacion estandar antes de calcular $\mathrm{Cov}(X)$.

Esto equivale a trabajar con la **matriz de correlacion** en lugar de la de covarianza.

```
mtcars_sm <- mtcars %>% select(mpg, hp, wt)
pc_1      <- prcomp(mtcars_sm, scale=TRUE)
round(pc_1$rotation, 3)  # cargas (loadings)
```

```
##        PC1    PC2    PC3
## mpg -0.603  0.163 -0.781
## hp   0.551  0.793 -0.260
## wt   0.576 -0.587 -0.568
```

Cada columna es un vector w_j: la combinacion lineal de las variables originales que define el componente j.

Varianza explicada

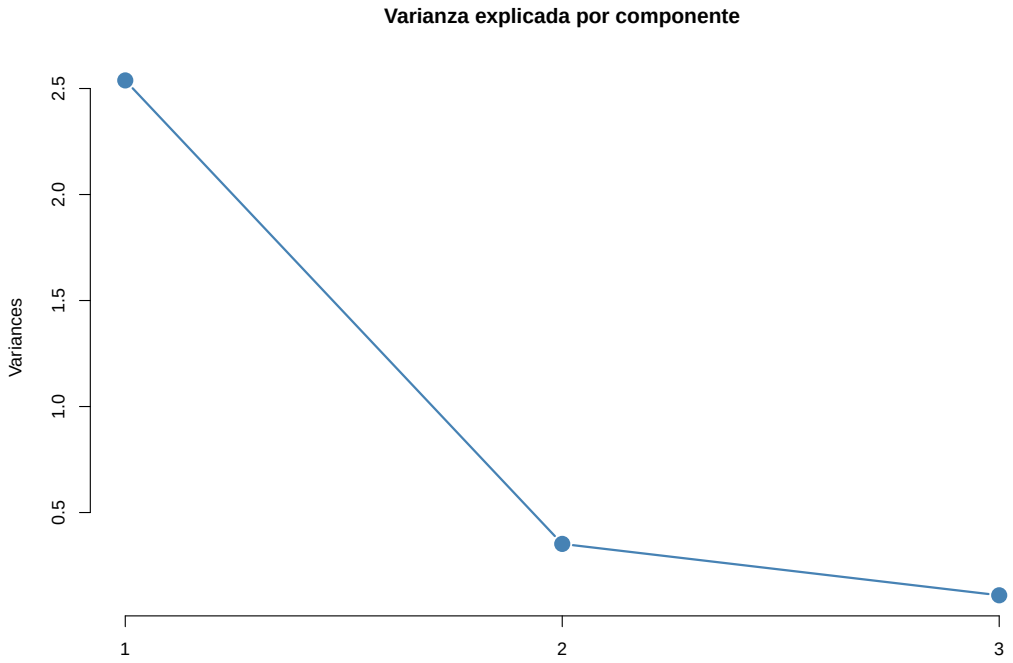
```r
summary(pc_1)
```

```
## Importance of components:
##                           PC1    PC2     PC3
## Standard deviation     1.5932 0.5934 0.33123
## Proportion of Variance 0.8461 0.1174 0.03657
## Cumulative Proportion  0.8461 0.9634 1.00000
```

El PC1 explica el 92% de la varianza total: casi toda la informacion de las 3 variables se puede resumir en una sola dimension latente.

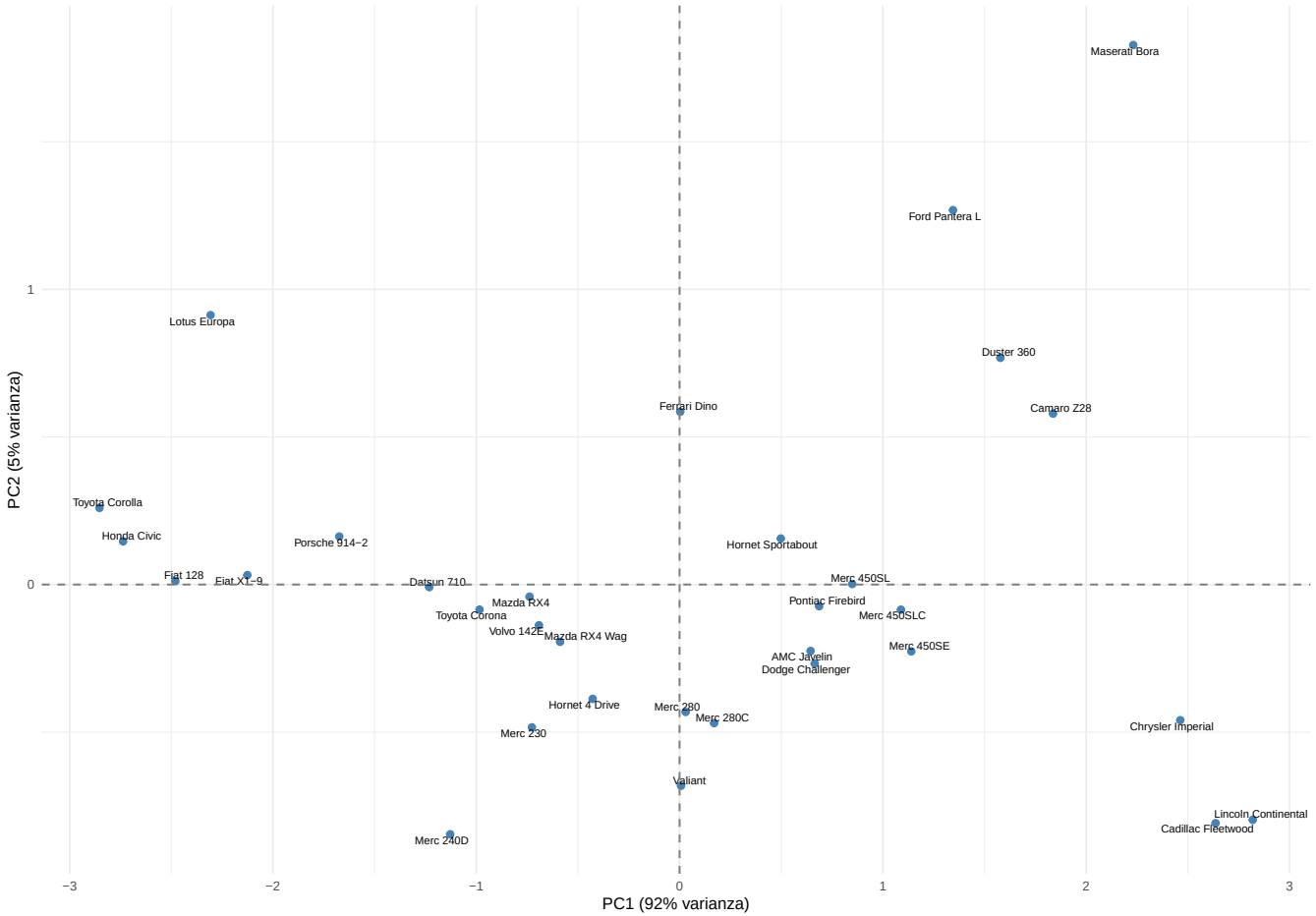
Screeplot: cuantos componentes retener?

```
screeplot(pc_1, col="steelblue", pch=16,
          type="lines", cex=2, lwd=2,
          main="Varianza explicada por componente")
```



El criterio clasico es retener los componentes antes del "codo" (punto donde la varianza adicional explicada se vuelve marginal).

Plano factorial: la variable latente visualizada



Interpretacion de los componentes

```r
# Correlacion de las variables originales con los PCs
cor(mtcars_sm, pc_1$x) |> round(3)
```

```
##        PC1    PC2    PC3
## mpg -0.961  0.097 -0.259
## hp   0.878  0.470 -0.086
## wt   0.918 -0.348 -0.188
```

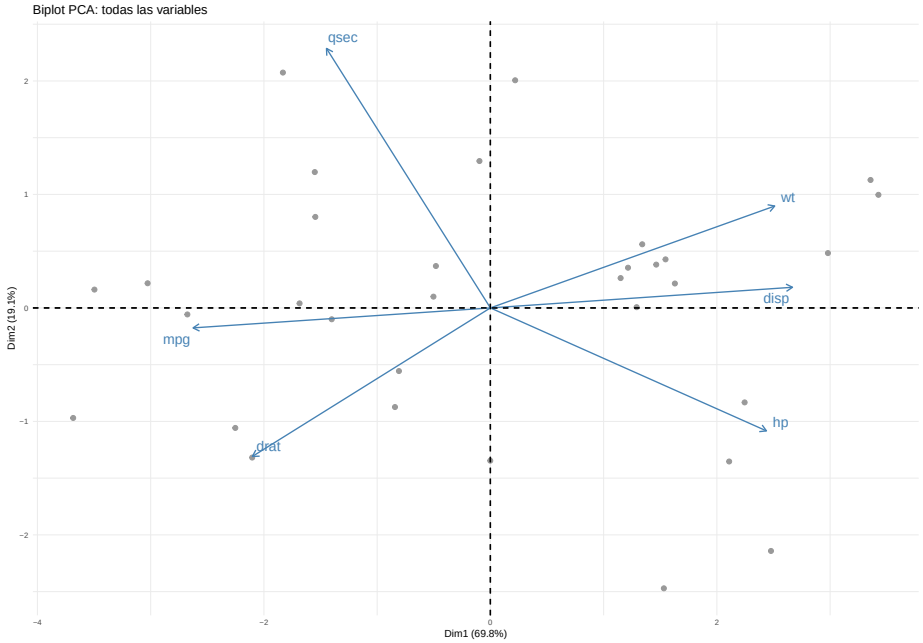
PC1 correlaciona negativamente con mpg y positivamente con hp y wt: es una dimension de **"tamano/potencia"**.

PC2 captura la variacion residual ortogonal a PC1.

Los vehiculos de la izquierda (PC1 negativo) son livianos y eficientes; los de la derecha son grandes y potentes. PC1 es la **variable latente de "tipo de vehiculo"**: no existe en los datos originales, pero los resume.

PCA con todas las variables

```
pc_full <- prcomp(
  mtcars %>% select(-car,-am,-vs,-gear,-carb,-cyl),
  scale=TRUE)
fviz_pca_biplot(pc_full,
                repel=TRUE,
                label="var",
                col.var="steelblue",
                col.ind="grey60",
                title="Biplot PCA: todas las variables") +
  theme_minimal(base_size=8)
```



El biplot muestra simultaneamente los individuos (puntos) y las variables (flechas) en el espacio de las dos primeras dimensiones latentes.

Puente: de MLE a variables latentes

Algoritmo EM: Mixtura de gaussianas

Algoritmo EM: caso general

PCA: variable latente continua

ACM: variable latente categorica

Sintesis y cierre

Referencias

ACM: variable latente categorica

Motivacion: cuando los datos no son numericos

PCA funciona con variables **continuas** porque necesitamos calcular medias, varianzas y covarianzas.

Pero muchos datos son **categoricos**:

- Encuestas: "Siempre / A veces / Nunca".
- Perfil de cliente: "Nivel educativo / Estrato / Tipo de producto".
- Biologia: presencia/ausencia de caracteristicas.

El **Analisis de Correspondencias Multiples (ACM)** es el analogo de PCA para variables categoricas.

ACM como variable latente

Al igual que PCA, ACM busca dimensiones latentes que resuman la estructura de los datos. La diferencia es que las "covarianzas" entre categorias se reemplazan por **chi-cuadrados de asociacion**.

El dataset: habitos de consumo de te

Encuesta a 300 consumidores de te con 6 preguntas categoricas:

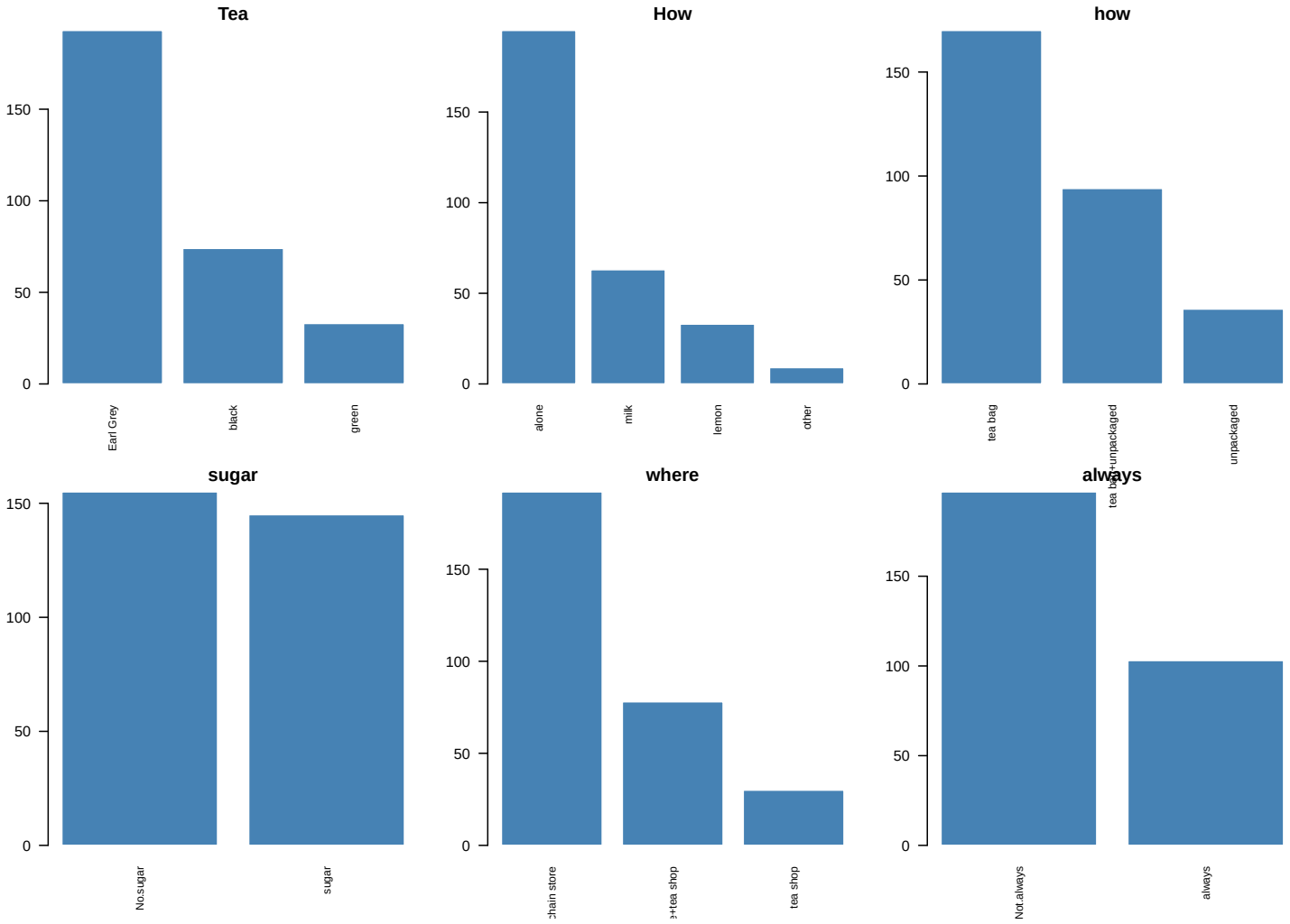
```
library(FactoMineR)
data(tea)
tea_hab <- tea[, c("Tea","How","how","sugar",
                   "where","always")]
str(tea_hab)
```

```
## 'data.frame':    300 obs. of  6 variables:
##  $ Tea   : Factor w/ 3 levels "black","Earl Grey",..: 1 1 2 2 2 2 2 1 2 1 ...
##  $ How   : Factor w/ 4 levels "alone","lemon",..: 1 3 1 1 1 1 1 3 3 1 ...
##  $ how   : Factor w/ 3 levels "tea bag","tea bag+unpackaged",..: 1 1 1 1 1 1 1 1 2 2 ...
##  $ sugar : Factor w/ 2 levels "No.sugar","sugar": 2 1 1 2 1 1 1 1 1 1 ...
##  $ where : Factor w/ 3 levels "chain store",..: 1 1 1 1 1 1 1 1 2 2 ...
##  $ always: Factor w/ 2 levels "always","Not.always": 2 2 2 2 1 2 2 2 2 2 ...
```

- `Tea`: tipo de te (Earl Grey, Verde, Otro).
- `How`: como lo prepara (solo, con leche, con limon. . .).
- `how`: en bolsita o suelto.
- `sugar`: con o sin azucar.
- `where`: donde lo compra.
- `always`: lo toma siempre o no.

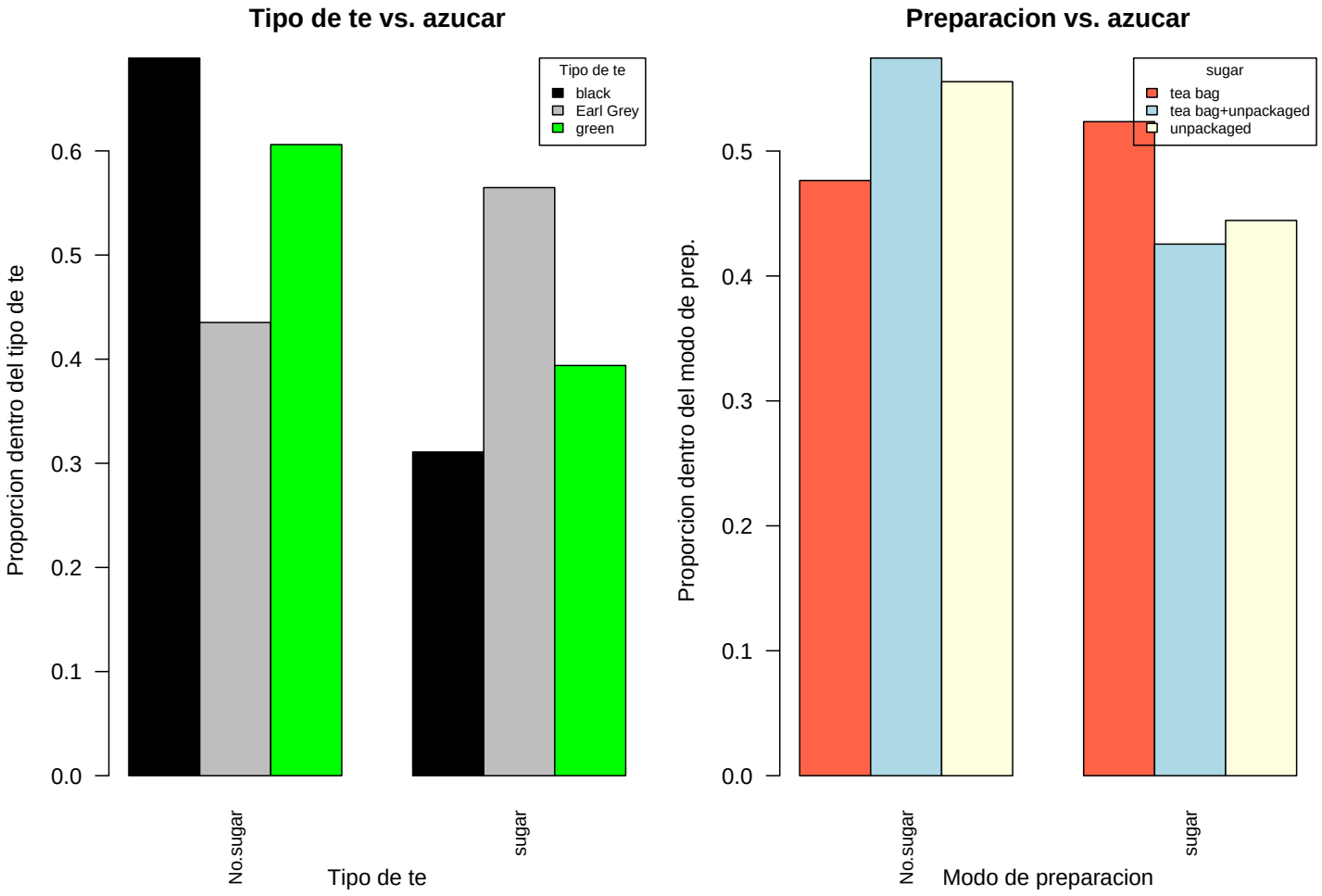
Descripcion univariada

Antes de cualquier analisis multivariado, exploramos cada variable por separado:



Descripcion bivariada: asociacion entre variables

Antes del ACM es util explorar las asociaciones entre pares.



El te Earl Grey y el te en bolsita tienden mas al azucar. El ACM capturara estas asociaciones en dimensiones latentes.

La matriz indicadora Z

El ACM construye la **matriz indicadora** $Z \in \{0,1\}^{n \times J}$ mediante codificacion *one-hot*: una columna por cada categoria de cada variable. Con Q variables de $J_1, \ldots, J_Q$ categorias: $J = J_1 + \cdots + J_Q$ columnas en total.

Cada fila tiene exactamente Q unos: uno por variable respondida.

```r
# Z para las primeras 5 personas, 2 variables: Tea (3 cat) + how (2 cat)
# --> J = 3 + 2 = 5 columnas
Z_ej <- model.matrix(~ Tea + how - 1, data=tea_hab[1:5,])
colnames(Z_ej) <- c("EarlGrey","Green","Other","bag","unpackaged")
Z_ej
```

```
##   EarlGrey Green Other bag unpackaged
## 1        1     0     0   0          0
## 2        1     0     0   0          0
## 3        0     1     0   0          0
## 4        0     1     0   0          0
## 5        0     1     0   0          0
## attr(,"assign")
## [1] 1 1 1 2 2
## attr(,"contrasts")
## attr(,"contrasts")$Tea
## [1] "contr.treatment"
##
## attr(,"contrasts")$how
## [1] "contr.treatment"
```

Cada fila suma exactamente 2 (un 1 en Tea, un 1 en how).

La distancia chi-cuadrado: intuicion

La **distancia chi-cuadrado** compara los perfiles de dos individuos pero ponderando por la rareza de cada categoria:

$$d_\chi^2(i, i') = Q \sum_{j=1}^{J} \frac{(z_{ij}/Q - z_{i'j}/Q)^2}{f_j}$$

donde $f_j = z_{\bullet j}/n$ es la frecuencia marginal de la categoria j.

Por que ponderar por f_j? Las categorias raras son mas informativas. Si dos personas comparten una categoria que solo tiene el 5% de frecuencia, eso dice mucho mas de su similitud que compartir una del 80%.

Ejemplo: imagine solo 2 categorias, "bolsita" (80%) y "suelto" (20%). Si dos personas toman te suelto (raro), su diferencia al cuadrado se divide por 0.20: la penalizacion es grande, haciendo que la distancia sea pequena (se parecen mucho en algo inusual). Dos personas que difieren en "suelto" versus "bolsita" tienen la diferencia dividida por un numero pequeno (0.20), ampliando esa diferencia.

La matriz de Burt

La **matriz de Burt** $B = Z^T Z \in \mathbb{R}^{J \times J}$ es una matriz simetrica de $J \times J$ bloques:

- **Diagonal de bloques:** tablas de frecuencia de cada variable con ella misma (frecuencias marginales en la diagonal).
- **Fuera de la diagonal:** tablas de contingencia entre cada par de variables (cuantos individuos eligieron la categoria j de una variable Y la categoria k de otra).

```r
# El bloque Tea x how de B es exactamente la tabla de contingencia:
table(tea_hab$Tea, tea_hab$how)
```

```
##
##             tea bag tea bag+unpackaged unpackaged
##   black          36                 23         15
##   Earl Grey     117                 60         16
##   green          17                 11          5
```

Esto es exactamente el bloque $B_{\text{Tea,how}}$ de la matriz de Burt. La matriz B compila **todas** las tablas de contingencia de todos los pares de variables en una sola estructura $J \times J$.

La funcion objetivo del ACM

El ACM busca dimensiones latentes que maximicen la **inercia** capturada, el analogo categorico de la varianza.

- La idea es simple: ACM aplica **SVD** a la matriz indicadora Z normalizada, exactamente como PCA aplica la descomposicion espectral a $\mathrm{Cov}(X)$.

La SVD produce:

- **Scores de individuos** $u_i \in \mathbb{R}^k$: posicion de cada persona en el plano latente.
- **Scores de categorias** $v_j \in \mathbb{R}^k$: posicion de cada categoria en el mismo plano.

Los scores se construyen de forma que individuos que eligieron categorias similares queden cerca, y categorias elegidas por los mismos individuos queden cerca.

Ajuste del ACM en R

```r
acm_res <- MCA(tea_hab, graph=FALSE)
# Varianza explicada
round(acm_res$eig[1:4, ], 3)
```

```
##       eigenvalue percentage of variance cumulative percentage of variance
## dim 1      0.280                 15.260                            15.260
## dim 2      0.258                 14.059                            29.319
## dim 3      0.220                 12.008                            41.326
## dim 4      0.188                 10.251                            51.577
```

- La inercia (varianza en sentido ACM) explicada por cada dimension. Es el analogo al porcentaje de varianza en PCA.
- Se retienen las primeras k dimensiones que concentran la mayor inercia.

De donde vienen los scores de individuos: la doble proyeccion

Los **scores de individuos** se calculan como el promedio ponderado de los scores de las categorias que eligieron:

$$u_i = \frac{1}{Q} \sum_{j:\, z_{ij}=1} v_j$$

Un individuo queda en el centroide de las categorias que respondio. Esto hace que el plano tenga una interpretacion directa: individuos similares (que eligieron categorias similares) quedan cerca.

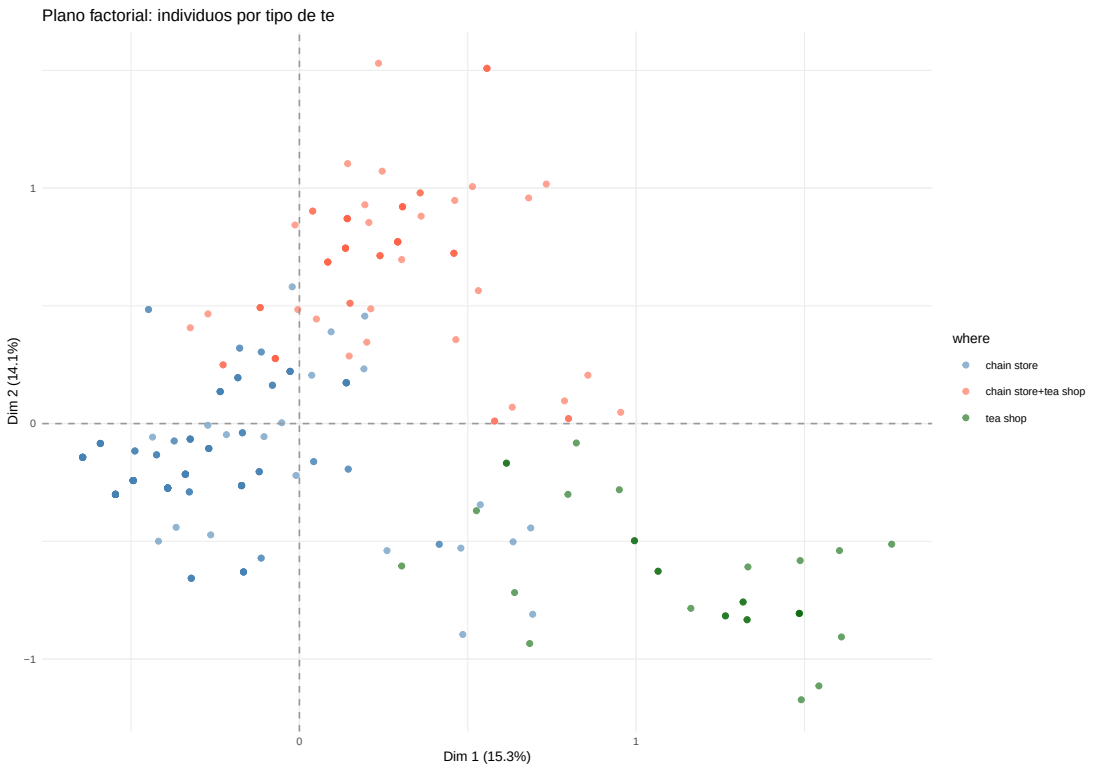
De donde vienen los scores de categorias: la doble proyeccion

Los **scores de categorias** se calculan analogamente como el centroide de los individuos que las eligieron:

$$v_j = \frac{n}{z_{\bullet j}} \sum_{i:\, z_{ij}=1} u_i$$

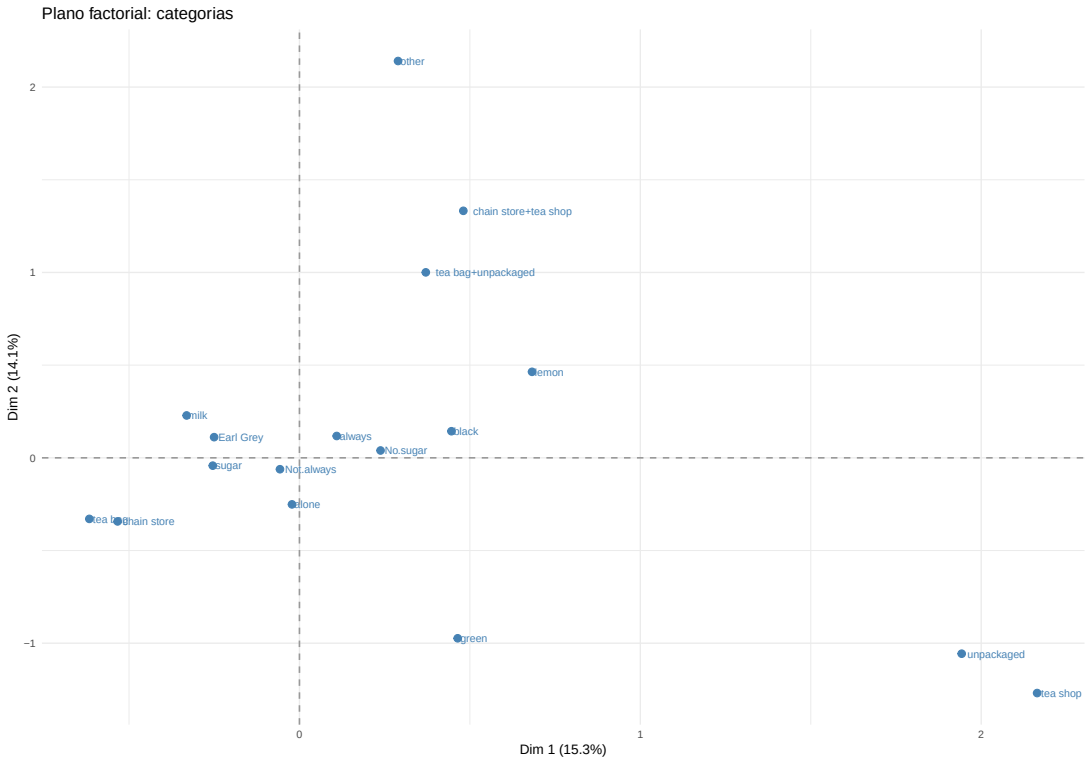
Esta dualidad entre individuos y categorias es la esencia del Analisis de Correspondencias.

Plano factorial del ACM: individuos



Cada punto es un consumidor. Los colores muestran el tipo de te (lugar). Los que consumen de forma similar quedan cerca en el plano.

Plano factorial del ACM: categorias



Categorias cercanas aparecen juntas en los mismos individuos. Categorias opuestas al origen definen extremos opuestos del eje.

ACM vs. PCA: la analogia completa

Aspecto	PCA	ACM
Tipo de datos	Continuos	Categoricos
Matriz de entrada	$X \in \mathbb{R}^{n \times p}$	Indicadoras $\in \{0,1\}^{n \times J}$
Distancia usada	Euclidiana	Chi-cuadrado
Medida de dispersion	Varianza	Inercia
Componentes	Combinaciones lineales de vars.	Dimensiones de asociacion
Variable latente	Score continuo	Score continuo en espacio chi-cuadrado
Criterio de retencion	Screeplot / regla del codo	Inercia acumulada
Implementacion en R	`prcomp()`, `PCA()`	`MCA()`, `ca()`

Puente: de MLE a variables latentes

Algoritmo EM: Mixtura de gaussianas

Algoritmo EM: caso general

PCA: variable latente continua

ACM: variable latente categorica

Sintesis y cierre

Referencias

Sintesis y cierre

Marco unificador: variables latentes

Los tres metodos vistos comparten la misma logica de fondo:

El principio de la variable latente

Los datos observados X son manifestaciones de una estructura subyacente Z que no podemos medir directamente. El objetivo es **descubrir** Z a partir de X.

Metodo	X observado	Z latente	Como se descubre Z
EM/Mixtura	Datos y_i	Grupo Δ_i	Probabilidad posterior
PCA	p vars. cont.	k scores	Vectores propios de Cov
ACM	p vars. cat.	k scores	SVD de matriz de indicadoras

Hilo conductor del curso

S1: MCO – regularizacion Ridge/Lasso.

S2: Validacion – CV, LOOCV, Bootstrap.

S3: No-linealidad – Splines, GAMs.

S4: Localidad – KDE, LOESS, kernels.

S5: Respuestas no continuas – GLM, Logistica.

S6: Variables latentes – EM, PCA, ACM.

Los seis temas comparten la estructura comun del aprendizaje estadistico: modelo, estimacion, seleccion de complejidad, validacion.

> ***Proxima sesion:*** *datos no estructurados – texto, imagenes, representaciones latentes profundas.*

Guia practica: cuando usar cada metodo

Situacion	Metodo
Datos con subgrupos desconocidos	EM / Mixtura gaussiana
Muchas vars. continuas correlacionadas	PCA
Muchas vars. categoricas	ACM
Vars. continuas y categoricas mixtas	FAMD (`FactoMineR`)
Subgrupos en espacio de PCs	K-means sobre PCs
Variable respuesta continua	Regresion sobre PCs (PCR)
Variable respuesta binaria	Logistica sobre PCs

Referencias

Bibliografia

(1) Hastie, T., Tibshirani, R., Friedman, J. *The Elements of Statistical Learning*. Springer. (Cap. 8: Model Inference and Averaging; Cap. 14: Unsupervised)

(2) James, G., Witten, D., Hastie, T., Tibshirani, R. *An Introduction to Statistical Learning with R*, 2a ed. Springer. (Cap. 12: Unsupervised Learning)

(3) Greenacre, M., Blasius, J. *Multiple Correspondence Analysis and Related Methods*. CRC Press, 2006.

(4) Le Roux, B., Rouanet, H. *Multiple Correspondence Analysis*. Sage Publications, 2010.

Paquetes en R:

- `mixtools::normalmixEM()` – Mixtura de gaussianas via EM.
- `stats::prcomp()` – PCA.
- `FactoMineR::PCA()`, `FactoMineR::MCA()` – PCA y ACM avanzado.
- `factoextra` – Visualizacion de PCA y ACM.